

Table des matieres

1. Resume executif	2
2. Audit des routes API	2
3. Analyse de securite	5
4. Audit du frontend	5
5. Problemes d'architecture	7
6. Propositions d'evolutions et ameliorations	8
7. Plan de priorites et feuille de route	11
8. Dependances inutilisees	11
9. Conclusion	12

1. Resume executif

Le present rapport constitue un audit technique complet du projet **RestoProGN**, une application web de gestion de restaurant developpee avec Next.js, Prisma ORM et SQLite. Ce projet, destine au restaurant **KFM Delice** a Conakry (Guinee), vise a devenir une solution white-label adaptable a d'autres etablissements de restauration en Guinee et en Afrique de l'Ouest. L'audit couvre l'ensemble des couches de l'application : architecture, base de donnees, API, securite, interface utilisateur, et experience client.

L'analyse revele que le projet possede une base fonctionnelle solide avec 13 onglets d'administration, un systeme de caisse POS, un portail client, et un suivi de livraison. Cependant, des lacunes critiques en matiere de securite (absence totale d'authentification, mots de passe en clair, donnees exposees), une architecture monolithique (fichier unique de 3 057 lignes), et plusieurs fonctionnalites incompletes (carte de livraison, paiement en ligne, notifications) limitent significativement sa mise en production. Ce rapport identifie 12 problemes de securite critiques, 25 fonctionnalites manquantes ou defectueuses, et 13 problemes d'architecture, accompagnees de recommandations priorisees pour chaque axe d'amelioration.

1.1 Tableau de bord de l'audit

Categorie	Score	Commentaire
Completude CRUD	7/10	La plupart des ressources ont un CRUD complet ; avis manquent PATCH ; reservations manquent DELETE
Multi-tenant	6/10	Schema supporte ; GET/POST filtrent par restaurant ; mais Admins et Customers sans restaurantId
Securite	1/10	Zero auth, mots de passe en clair, identifiants exposes, endpoint seed non protege
Coherence Frontend-Backend	5/10	La plupart des boutons fonctionnent ; points de fidelite, carte live, commande en ligne sont UI-only
Architecture	2/10	Monolithe 3000+ lignes, pas de state management, pas de middleware, SQLite
Production-ready	1/10	Non deployable : auth, hachage, sessions, refactoring indispensables

2. Audit des routes API

L'application dispose de 19 endpoints API repartis sur les ressources principales du systeme. L'audit a revele que si la plupart des routes offrent les operations CRUD de base, aucune ne dispose de verification d'authentification, et plusieurs presentent des bugs fonctionnels ou des failles de securite. Le tableau ci-dessous synthetise l'etat de chaque route, suivi d'une analyse detaillee des problemes les plus critiques.

2.1 Synthèse des routes API

Route	Methodes	Etat	Probleme principal
/api/route	GET	Inutile	Retourne "Hello world" ; code mort
/api/login	POST	Partiel	Pas de session/JWT ; mot de passe en clair
/api/customer-login	POST	Partiel	Meme probleme que login admin
/api/customer-register	POST	Partiel	Mot de passe en clair ; pas de validation email
/api/menu	GET/POST/PATCH/DELETE	Bon	PATCH/DELETE ne verifient pas restaurantId
/api/orders	GET/POST/PATCH	Moyen	Bug driver status ; pas de validation de transition
/api/reservations	GET/POST/PATCH	Moyen	Pas de DELETE ; PATCH = status uniquement
/api/customers	GET/POST/PATCH/DELETE	Critique	Pas multi-tenant ; changement mdp sans verification
/api/drivers	GET/POST/PATCH/DELETE	Bon	PATCH/DELETE sans verification restaurantId
/api/driver-location	GET/PATCH	Critique	Pas d'auth ; n'importe qui peut modifier la position GPS
/api/staff	GET/POST/PATCH/DELETE	Bon	Pas d'auth ; salaires visibles
/api/admins	GET/POST/PATCH/DELETE	Critique	GET expose les mots de passe ; pas multi-tenant
/api/reviews	GET/POST/DELETE	Moyen	Pas de PATCH ; pas de validation rating
/api/stats	GET	Moyen	Surcharge memoire ; tout charge en JS
/api/expenses	GET/POST/PATCH/DELETE	Bon	Standard ; pas d'auth
/api/invoices	GET/POST/PATCH/DELETE	Bon	Pas de generation PDF
/api/quotes	GET/POST/PATCH/DELETE	Bon	Pas de generation PDF
/api/tracking	GET	Moyen	Pas d'auth ; n'importe qui peut tracer

/api/seed	POST	Critique	Non protege ; reinitialise la BDD ; appelle au chargement
-----------	------	----------	---

2.2 Problemes critiques des API

2.2.1 Absence totale d'authentification

Aucune des 19 routes API ne verifie l'identite de l'appelant. Il n'existe ni session, ni jeton JWT, ni cookie d'authentification. Le systeme de login actuel se contente de verifier email et mot de passe, puis retourne l'objet utilisateur sans creer de session persistante. Cela signifie qu'un visiteur anonyme peut consulter les salaires du personnel, les mots de passe des administrateurs, modifier les positions GPS des livreurs, ou reinitialiser la base de donnees entiere via l'endpoint /api/seed. Ce probleme est le plus critique de l'ensemble du projet et doit etre resolu en priorite absolue avant toute mise en production.

2.2.2 Mots de passe en clair

Les mots de passe des administrateurs et des clients sont stockes en texte clair dans la base SQLite et compares directement sans aucun hachage. Les routes /api/admins et /api/customers retournent les mots de passe dans les reponses GET, ce qui les rend accessibles a tout utilisateur ou script accedant a l'API. La correction necessite l'integration de bcrypt ou argon2 pour le hachage, la migration des mots de passe existants, et l'exclusion systematique du champ password des reponses API.

2.2.3 Changement de mot de passe sans verification

L'endpoint PATCH /api/customers accepte un changement de mot de passe sans exiger la verification de l'ancien mot de passe. N'importe qui connaissant l'identifiant d'un client peut modifier son mot de passe et prendre le controle de son compte. La page de profil client dans le frontend envoie directement {id, password: nouveauMdp} sans aucune verification de securite.

2.2.4 Endpoint seed non protege

La route POST /api/seed permet de reinitialiser la totalite de la base de donnees. Elle est appelee automatiquement a chaque chargement de la page d'accueil via un useEffect dans le composant Home. N'importe quel visiteur declenche ainsi potentiellement la reinitialisation de la base, ecrasant toutes les donnees existantes. Cet endpoint doit etre supprime du code de production ou protege par une authentification administrateur stricte.

2.2.5 Isolation multi-tenant incomplete

Le modele Customer ne possede pas de champ restaurantId, ce qui signifie que tous les clients de tous les restaurants partagent le meme espace de donnees. Le modele Admin souffre du meme probleme. En consequence, dans un contexte multi-restaurant, un client du restaurant A pourrait voir les donnees du restaurant B, et un administrateur d'un restaurant pourrait gerer les comptes d'un autre etablissement. Les routes PATCH et DELETE ne verifient jamais l'appartenance restaurantId, permettant des modifications transversales entre etablissements.

3. Analyse de securite

L'analyse de securite revele un niveau de protection insuffisant pour une application manipulant des donnees financieres, des informations personnelles (noms, telephones, adresses), et des identifiants de connexion. Le tableau ci-dessous recense l'ensemble des vulnerabilites identifiees, classees par niveau de severite, avec des recommandations de correction pour chacune d'entre elles.

Severite	ID	Vulnerabilite	Recommandation
CRITIQUE	S1	Zero authentication API	Implementer JWT + middleware Next.js
CRITIQUE	S2	Mots de passe en clair	Integrer bcrypt pour hachage
CRITIQUE	S3	Mots de passe admin exposes dans GET /api/admins	Exclure password des reponses API
CRITIQUE	S4	Changement mdp sans verification ancien mdp	Exiger ancien mdp avant changement
CRITIQUE	S5	Endpoint /api/seed non protege et auto-appel	Protéger ou supprimer en production
HAUT	S6	Pas de protection CSRF	Ajouter tokens CSRF
HAUT	S7	Pas de rate limiting	Ajouter rate limiting sur login et CRUD
MOYEN	S8	Identifiants affiches sur page de login	Supprimer les identifiants demo visibles
MOYEN	S9	Pas de validation/sanitisation des entrees	Ajouter zod pour validation schema
MOYEN	S10	Pas d'autorisation par role cote serveur	Implementer middleware de verification de role
MOYEN	S11	Isolation client cote frontend uniquement	Filtrer cote serveur par restaurantId
BAS	S12	CORS non configure explicitement	Configurer CORS pour les domaines autorises

4. Audit du frontend

L'interface utilisateur est entierement contenue dans un fichier unique (page.tsx) de 3 057 lignes, regroupant 19 composants, l'ensemble de la logique de routage, les appels API, et la gestion d'etat. Le site public comprend 6 sections (navbar, hero, menu, reservation, avis, a propos), le tableau de bord administrateur comporte 13 onglets, et le portail client dispose de 5 onglets. L'audit a identifie plusieurs

fonctionnalites presentees dans l'interface mais non fonctionnelles ou incompletes.

4.1 Composants et onglets

Composant	Categorie	Statut	Remarque
AdminLogin	Auth	Fonctionnel	Identifiants demo visibles
CustomerLogin	Auth	Fonctionnel	Pas de session persistante
CustomerRegister	Auth	Fonctionnel	Pas de validation email
CustomerAccount	Client	Partiel	Filtrage donnees cote client uniquement
DeliveryTrackingPage	Client	UI seule	Pas de carte ; timeline sans carte GPS
AdminDashboard	Admin	Partiel	13 onglets ; rechargement massif a chaque action
PublicNavbar	Public	Fonctionnel	-
HeroSection	Public	Partiel	Compteur avis (327) code en dur
MenuSection	Public	Fonctionnel	Commande via WhatsApp uniquement
ReservationSection	Public	Fonctionnel	Pas de verification date passee

4.2 Fonctionnalites defectueuses ou manquantes

Categorie	Fonctionnalite	Detail du probleme
Auth	Gestion de sessions (JWT/cookies)	Aucune session persistante ; rafraichissement = deconnexion
Auth	Hachage des mots de passe	Tous les mots de passe sont en clair
Auth	Reinitialisation de mot de passe	Aucun flux de recuperation de compte
Commandes	Commande en ligne sur le site public	Lien WhatsApp uniquement, pas de panier/checkout
Commandes	Machine a etats des statuts de commande	Transition libre entre n'importe quels statuts
Livraison	Carte GPS en temps reel	Champs lat/lng existent mais aucune carte affichee
Livraison	Calcul ETA reel	Toujours fixe a maintenant + 30 minutes
Documents	Generation PDF factures/devis	Absent ; pas d'endpoint de generation
Menu	Drag-and-drop pour reorganisation	@dnd-kit dans les dependances mais non implemente

Notifications	Push/email/SMS	Aucun systeme de notification
Multi-restaurant	Selecteur de restaurant	Prend toujours le premier restaurant trouve
Rapports	Export CSV/PDF	Aucun export de donnees
Recherche	Recherche plein texte	Filtrage uniquement cote client ou match exact
Images	Upload images pour le menu	Chaines URL uniquement ; pas d'upload
i18n	Internationalisation	next-intl dans les dependances mais non configure
Analytics	Graphiques sur le dashboard	recharts dans les dependances mais non utilise
Paielements	Integration Orange Money / MTN Money	Selection UI uniquement ; aucun traitement reel
Temps reel	WebSocket pour mises a jour live	Polling 5s ; WebSocket non integre
Fidelite	Logique d'accumulation des points	Points affiches mais jamais accumules automatiquement
POS	Commandes mises en attente/rappelees	Cree une commande mais pas de rappel possible
POS	Calcul de la taxe	Champ taxe toujours a 0 dans les soumissions POS

5. Problemes d'architecture

L'architecture actuelle du projet presente des faiblesses structurelles qui impactent la maintenabilite, les performances, et la capacite a evoluer vers une solution multi-restaurant robuste. Les problemes identifies ci-dessous necessitent une refonte progressive plutot que des corrections ponctuelles.

ID	Probleme	Description detaillee
A1	Monolithe de 3 057 lignes	Tout le code dans page.tsx ; 19 composants, tout l'etat, tous les appels API. Inmaintenable et impossible a tester unitairement.
A2	Pas de state management global	Zustand est dans les dependances mais non utilise. Tout l'etat est local via useState, sans persistance ni partage entre composants.
A3	Pas de middleware d'authentification	Next-auth est dans les dependances mais non integre. Aucun middleware Next.js ne protege les routes API ou les pages.
A4	SQLite en production	Base de donnees fichier unique non concue pour les acces concurrents. Inadapte pour un usage multi-utilisateurs en production.
A5	Pas de migrations de base de donnees	Utilise prisma db push au lieu de prisma migrate. Pas d'historique de migrations, pas de seed structure.

A6	Pas d'error boundaries React	Un crash d'un composant fait tomber l'application entiere sans possibilite de recuperation.
A7	Sur-requetage API	loadData() charge 11 endpoints simultanement a chaque modification, meme pour une simple mise a jour de statut.
A8	Auth cote client uniquement	L'etat d'authentification est un useState. Rafraichir la page deconnecte l'utilisateur.
A9	Donnees restaurant en dur	const RESTO = { name: "KFM Delice" } code en dur au lieu de venir de la base de donnees.
A10	Dependances inutilisees	next-auth, next-intl, zustand, react-query, react-table, @dnd-kit, react-hook-form, zod, sharp, @mdxeditor/editor sont dans package.json mais non utilises.
A11	Pas de versioning API	Toutes les routes sont a /api/ressource sans prefixe de version.
A12	Pas de validation de schema	Aucune validation des requetes entrantes (zod disponible mais non utilise).
A13	Pas de tests	Aucun test unitaire, integration, ou E2E n'existe dans le projet.

6. Propositions d'evolutions et ameliorations

Les evolutions proposees sont classees par priorite en trois vagues : la premiere vague concerne les corrections indispensables pour la securite et la stabilite ; la deuxieme vague porte sur les fonctionnalites a forte valeur ajoutee pour les restaurants guineens ; la troisieme vague concerne les ameliorations architecturales a long terme.

6.1 Vague 1 : Securite et stabilite (Priorite critique)

E1 - Authentification JWT avec middleware Next.js

Implementer un systeme d'authentification complet base sur les JSON Web Tokens (JWT) avec un middleware Next.js qui protege toutes les routes API. Le middleware doit verifier la validite du token, extraire l'identite de l'utilisateur et son role, et injecter ces informations dans la requete. Les tokens doivent avoir une duree de vie limitee (1 heure pour l'access token, 7 jours pour le refresh token) avec un mecanisme de rotation des refresh tokens pour securiser les sessions longue duree.

E2 - Hachage des mots de passe avec bcrypt

Integrer la bibliotheque bcrypt pour le hachage de tous les mots de passe (administrateurs et clients). Le sel doit etre genere automatiquement avec un cout factor d'au moins 12. Les mots de passe existants en clair doivent etre migres lors d'un script de migration one-shot. Le champ password doit etre

systematiquement exclu des reponses API via select dans les requetes Prisma.

E3 - Protection de l'endpoint seed et suppression des identifiants visibles

L'endpoint /api/seed doit etre desactive en production ou protege par une authentication administrateur. L'appel automatique dans le useEffect du composant Home doit etre conditionne a l'environnement de developpement uniquement. Les identifiants de demo affiches sur les pages de login doivent etre remplaces par un lien vers une documentation interne accessible uniquement aux administrateurs.

E4 - Validation des entrees avec Zod

Utiliser la bibliotheque Zod (deja presente dans les dependances) pour valider le schema de toutes les requetes entrantes sur chaque route API. Chaque endpoint doit definir un schema Zod strict pour ses parametres et son corps de requete, et rejeter automatiquement toute requete ne conformant pas au schema attendu. Cela protege contre les injections, les donnees corrompues, et les abus de l'API.

6.2 Vague 2 : Fonctionnalites a forte valeur ajoutee

E5 - Carte de livraison en temps reel avec Leaflet

Integrer la bibliotheque Leaflet avec les tuiles OpenStreetMap pour afficher une carte interactive montrant la position en temps reel des livreurs. Cote client, la carte doit afficher le trajet du livreur vers l'adresse de livraison avec une estimation du temps d'arrivee basee sur la distance. Cote restaurant, le dashboard doit afficher toutes les livraisons actives sur une carte centralisee avec des marqueurs colores par statut. Les mises a jour de position doivent utiliser Server-Sent Events (SSE) plutot que le polling actuel pour reduire la charge serveur et ameliorer la fluidite.

E6 - Systeme de commande en ligne complet

Remplacer le lien WhatsApp par un vrai systeme de commande en ligne sur le site public. Le client doit pouvoir parcourir le menu, ajouter des articles a un panier, choisir le type de commande (sur place, emporter, livraison), saisir son adresse de livraison, selectionner le mode de paiement, et confirmer sa commande. Le panier doit etre persistant (localStorage ou base de donnees pour les clients connectes), et le processus de checkout doit guider l'utilisateur etape par etape avec validation a chaque stade.

E7 - Integration paiement Orange Money / MTN Money

Developper une integration reelle avec les APIs de paiement mobile money disponibles en Guinee. Le flux doit inclure l'initialisation du paiement via l'API de l'operateur, la redirection ou l'envoi du prompt USSD au client, la reception du webhook de confirmation, et la mise a jour automatique du statut de la commande. En l'absence d'API officielle, implementer un flux semi-manuel ou le client envoie une capture d'ecran de confirmation, validee par le restaurant.

E8 - Notifications push et SMS

Implementer un systeme de notifications multi-canal : notifications push via le navigateur (Web Push API), SMS via une passerelle locale guineene (Orange/MTN), et emails pour les confirmations de reservation et de commande. Les notifications doivent etre declenchees automatiquement lors des changements de statut des commandes et reservations, et configurables par le client dans ses preferences.

E9 - Generation PDF des factures et devis

Ajouter un endpoint de generation PDF pour les factures et les devis, utilisant une bibliotheque comme puppeteer ou ReportLab pour creer des documents professionnels avec l'en-tete du restaurant, les details du client, le tableau des articles, les taxes, et les conditions de paiement. Les PDF doivent etre telechargeables depuis l'interface admin et envoyables par email au client en un clic.

E10 - Programme de fidelite automatise

Developper une logique complete d'accumulation et de remuneration des points de fidelite. Les points doivent etre credites automatiquement a chaque commande terminee (par exemple 1 point par 1 000 GNF depense), et le client doit pouvoir les utiliser comme remise lors d'une commande future. Le dashboard admin doit afficher les statistiques du programme (points distribues, points utilises, taux de retention) et permettre la configuration des regles d'accumulation.

6.3 Vague 3 : Refonte architecturale

E11 - Decomposition du monolithe en composants autonomes

Decouper le fichier page.tsx de 3 057 lignes en composants React autonomes, chacun dans son propre fichier, organises dans une structure de dossiers coherente : components/admin/, components/public/, components/customer/, components/shared/. Chaque composant doit gerer son propre etat et ses propres appels API, avec des props clairement definies. Utiliser React.lazy et Suspense pour le chargement differe des composants lourds (carte, dashboard admin).

E12 - State management global avec Zustand

Implementer Zustand (deja dans les dependances) pour gerer l'etat global de l'application : authentication, restaurant courant, panier, preferences utilisateur. Cela permet la persistance de l'etat entre les navigations, le partage de donnees entre composants sans prop drilling, et la rehydratation automatique depuis le localStorage pour survivre aux rafraichissements.

E13 - Migration vers PostgreSQL

Migrer de SQLite vers PostgreSQL pour supporter les acces concurrents, les transactions ACID completes, et les performances necessaires en production. Prisma rend cette migration relativement transparente : il suffit de changer la chaine de connexion dans DATABASE_URL et d'executer prisma migrate deploy. PostgreSQL offre egalement le support natif du type JSON pour le champ items des

commandes, et des index Full-Text Search pour la recherche dans le menu et les commandes.

E14 - Tests automatisés

Mettre en place une suite de tests couvrant les trois niveaux : tests unitaires avec Vitest pour les fonctions utilitaires et les composants isolés, tests d'intégration avec Testing Library pour les flux utilisateur critiques (login, commande, réservation), et tests E2E avec Playwright pour les parcours complets (de la commande à la livraison). Objectif initial : 80% de couverture sur les routes API et les composants critiques.

E15 - Dashboard analytique avec graphiques

Exploiter la bibliothèque Recharts (déjà dans les dépendances) pour ajouter des graphiques interactifs au dashboard admin : courbes de chiffre d'affaires journalier/hebdomadaire/mensuel, diagrammes en camembert des catégories de ventes, histogrammes des heures de pointe, et graphiques de tendance des avis clients. Ces visualisations sont essentielles pour que le restaurateur puisse prendre des décisions éclairées basées sur ses données réelles.

7. Plan de priorités et feuille de route

Le tableau ci-dessous présente la feuille de route recommandée pour les évolutions, organisée en quatre phases étalées sur environ 6 mois. Les phases sont ordonnées par priorité, chaque phase construisant sur les fondations de la précédente. Les estimations de durée sont indicatives et supposent un développeur à temps plein.

Phase	Objectif	Évolutions	Durée est.	Contenu
Phase 1	Securite critique	E1, E2, E3, E4	2-3 semaines	Auth JWT, hachage mdp, protection seed, validation Zod
Phase 2	Fonctionnalite s core	E5, E6, E10	4-5 semaines	Carte livraison, commande en ligne, fidelite automatisee
Phase 3	Integrations	E7, E8, E9	3-4 semaines	Paiement mobile money, notifications, PDF factures
Phase 4	Architecture	E11-E15	4-6 semaines	Decomposition, Zustand, PostgreSQL, tests, analytics

8. Dépendances inutilisées

Le fichier package.json contient un nombre significatif de dependances qui ne sont pas utilisees dans le code de l'application. Ces dependances augmentent inutilement la taille du bundle, le temps d'installation, et la surface d'attaque potentielle. Il est recommande de les retirer si elles ne sont pas prevues pour une utilisation prochaine, ou de les integrer effectivement dans le cadre des evolutions proposees.

Dependance	Usage prevu	Recommandation
next-auth	Authentification	E1 (remplacer par JWT custom + middleware)
next-intl	Internationalisation	Futur : support multilingue (FR, Soussou, Malinke, Poular)
zustand	State management	E12 (state management global)
@tanstack/react-query	Requetes API	Remplacer les fetch manuels par react-query
@tanstack/react-table	Tableaux de donnees	Tables admin avec tri/filtre/pagination
@dnd-kit/core + sortable	Drag and drop	E5 ou reorganisation du menu
react-hook-form	Formulaires	Remplacer les formulaires controles manuellement
zod	Validation de schema	E4 (validation API)
sharp	Traitement d'images	Upload et redimensionnement images menu
@mdxeditor/editor	Editeur de texte riche	Edition de descriptions de menu
recharts	Graphiques	E15 (dashboard analytique)

9. Conclusion

RestoProGN possede les fondations d'une solution de gestion de restaurant prometteuse pour le marche guineen. L'interface est moderne et couvre un large eventail de fonctionnalites : reservations, commandes, livraisons, caisse POS, facturation, et portail client. Cependant, l'application dans son etat actuel ne peut pas etre mise en production en raison de lacunes critiques en securite (absence d'authentification, mots de passe en clair, donnees exposees) et d'une architecture monolithique qui limite la maintenabilite et l'evolution.

Les corrections prioritaires (Phase 1) sont estimees a 2-3 semaines et sont un prerequis absolu avant tout deploiement. Les fonctionnalites a forte valeur ajoutee (Phase 2) - carte de livraison, commande en ligne, fidelite - transforment l'application d'un outil de gestion interne en une plateforme complete reliant le restaurant a ses clients. Les integrations (Phase 3) - paiement mobile money, notifications, PDF - adaptent l'application au contexte guineen et la rendent commercialement viable. Enfin, la refonte architecturale (Phase 4) assure la perennite et la scalabilite de la solution a mesure que le nombre de restaurants clients augmente.

Le potentiel commercial est reel : il n'existe pas aujourd'hui de solution complete et abordable pour les restaurants en Guinee. RestoProGN, avec les evolutions proposees, peut combler ce vide et devenir la reference du marche en proposant une solution SaaS multi-restaurant adaptee aux specificites locales (paiement mobile money, langues locales, infrastructure reseau). L'investissement necessaire est mesurable et les retours attendus sont significatifs.